Learning Plaintext–Ciphertext Relations using Machine Learning

Subject : Applied Cryptography

Arjun Prasad
MT25064

Videh Sharma
MT25052

Yash Verma
MT25092

Index

Abstract

1. Introduction
2. Background and Related Work
3. Dataset Generation
4. Input Representations
5. Machine Learning Models
6. Implementation
7. Results and Observations
8. Conclusion
9. References

Abstract

This report presents a comprehensive study on neural cryptanalysis, specifically targeting the problem of distinguishing reduced-round block ciphers from random permutations. We implement and evaluate three machine learning architectures — *Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), and Siamese Network* — across 13 ciphers: **ASCON-128, GIFT-128, PRESENT, SIMON, SPECK, RECTANGLE,GRAIN,SPARX,XOODYAK,SNOW,ROCCA,LEA,SKINNY**.Our distinguishers learn exploitable plaintext–ciphertext relations without explicit cryptanalytic rules, building on the framework pioneered by Gohr (2019). We evaluate multiple input representations and identify the maximum number of cipher rounds for which each model maintains a statistically significant advantage over random guessing.

1. Introduction

Classical cryptanalysis of symmetric-key ciphers relies on exploiting mathematical weaknesses — differential and linear propagation patterns, algebraic structures, and statistical biases — to distinguish a cipher from an ideal random function. These techniques require expert knowledge and extensive manual analysis. The emergence of deep learning has opened a new paradigm: *neural cryptanalysis*, where machine learning models automatically learn exploitable relations from data, without being hand-crafted with cryptographic insight.

The seminal work of Gohr (2019)[1] demonstrated that a carefully designed residual neural network could outperform classical differential distinguishers on SPECK-32/64, prompting significant interest in applying ML to lightweight ciphers. Lightweight ciphers — designed for constrained IoT and embedded environments — often operate with reduced round counts for performance, making them natural targets for reduced-round cryptanalysis.

This project addresses the following central question: ***given pairs of plaintexts with a fixed XOR difference and their corresponding ciphertexts under a reduced-round block cipher, can a machine learning model learn to distinguish the cipher from a random permutation?*** We answer this affirmatively across multiple ciphers and round reductions, implementing three architectures and experimenting with diverse input representations.

1.1 Objectives

- Implement a dataset generation pipeline for 13 lightweight block ciphers with configurable round reductions.
- Design and train three ML distinguishers: MLP, CNN, and Siamese Network.
- Evaluate on multiple input representations.
- Identify the maximum number of rounds for which each distinguisher achieves accuracy significantly above 50%.

- Compare model architectures and analyze the cryptographic implications of the learned representations.

2. Background and Related Work

2.1 Differential Cryptanalysis

Differential cryptanalysis, introduced by Biham and Shamir, exploits non-uniform distributions in output differences arising from fixed input differences. Given a plaintext pair (P, P') with P XOR P' = dP, the ciphertext difference dC = C XOR C' follows a non-uniform distribution for a real cipher but is nearly uniform for a random permutation. This statistical bias grows weaker as the number of cipher rounds increases, eventually disappearing at the full round count.

2.2 Neural Cryptanalysis

Gohr (2019) reformulated differential distinguishing as a binary classification task. Given ciphertext pairs (C, C'), a neural network predicts whether they were produced by a cipher (label 1) or a random permutation (label 0). The network implicitly learns differential distributions without being told about cryptographic structure. This approach has since been extended to numerous ciphers and has motivated the study of input representations, architectures, and the relationship between learned features and classical differential trails.

2.3 Ciphers Under Study

We study the following 13 lightweight ciphers, all widely deployed or proposed for constrained environments:

Cipher	Block Size	Key Size	Design Paradigm	Full Rounds	Description
ASCON-128	128-bit	128-bits	Sponge / ARX-like permutation	12	Sponge-based AEAD cipher with fast diffusion and strong permutation structure.
GIFT-128	128-bit	128-bits	SPN (bit-oriented)	40	Lightweight SPN cipher with bit permutation and nibble-wise S-box operations.
PRESENT	64-bit	80/128-bits	SPN (bit-oriented)	31	Ultra-lightweight SPN cipher with simple

Cipher	Block Size	Key Size	Design Paradigm	Full Rounds	Description
					S-box and slow diffusion.
SIMON	32–128-bit	64–256-bits	Feistel (ARX)	32–72	Feistel ARX cipher with very slow diffusion using AND-XOR-rotation.
SPECK	32–128-bit	64–256-bits	ARX	22–34	ARX cipher using modular addition and rotations with fast diffusion.
RECTANGLE	64-bit	80/128-bits	SPN (bit-slice friendly)	25	Bit-sliced SPN cipher optimized for hardware efficiency.
SPARX	64/128-bit	128-bits	ARX (LAX design)	24–40	ARX cipher combining parallel ARX boxes with linear mixing.
XOODYAK	384-bit	128-bits	Sponge / permutation-based	12	Sponge-based cipher using Xoodoo permutation with strong diffusion.
GRAIN-128	N/A(Stream)	128-bits	Stream cipher / LFSR+NLFSR	256 init	Stream cipher using LFSR and NFSR with keystream generation.
LEA	128-bits	128/192/256-bit	ARX (Add-Rotate-XOR)	24/28/32	Pure ARX cipher using 32-bit operations without S-boxes.
SKINNY	64/128-bits	64-384 bits	SPN (bit-slice friendly, tweakable	32–56	Tweakable SPN cipher with efficient S-box and MDS diffusion.
SNOW-Vi	N/A (stream)	128/256-bit	Stream cipher / LFSR + FSM (improved SNOW-V)	16 init	Stream cipher combining LFSR and AES-based FSM.
ROCCA	128-bit (state-based AEAD)	128/256-bit	AES-based / permutation (AEAD)	20 steps	AES-based AEAD cipher with fast avalanche and permutation structure.

3. Dataset Generation

Dataset generation follows the canonical differential distinguisher setup. For each cipher C and round count r, we generate N labeled samples split equally between cipher-generated and randomly-generated pairs.

3.1 Sampling Procedure

For cipher-generated samples (label y = 1):

- Sample a random plaintext P uniformly from {0,1}^n.
- Compute paired plaintext P' = P XOR dP, where dP is a fixed input difference.
- Sample a random key K uniformly from the key space.
- Compute ciphertext pairs (C, C') = (C_K^r(P), C_K^r(P')).

For random samples (label y = 0):

- Sample two independent ciphertexts C, C' uniformly at random from {0,1}^n.

The default input difference dP is set to the all-zeros-except-one-bit pattern (e.g., 0x80000000...0 for 128-bit ciphers), following common differential cryptanalysis conventions. The dataset is balanced with equal class proportions.

3.2 Dataset Parameters

Parameter	Value	Notes
Total samples (N)	1,00,000	50K cipher + 50K random per (cipher, round)
Train / Val / Test split	70% / 15% / 15%	Stratified by label
Input difference dP	Fixed, cipher-specific	Chosen to maximize differential bias at low rounds
Key generation	Fresh random key per sample	Prevents key-recovery leakage
Storage format	.npy arrays	X1, X2 (plaintexts+ciphertexts), y (labels)

3.3 Data Layout

Each sample is stored as two arrays X1 and X2, each of shape (N, 256) for 128-bit ciphers (128 plaintext bits + 128 ciphertext bits). For 64-bit ciphers, the shape is (N, 128). Labels y are stored as binary arrays of shape (N,).

4. Input Representations

The choice of input representation critically determines what statistical structure is exposed to the model. All representations are implemented once in `representation.py` and shared identically across MLP, CNN, and Siamese — there is no per-model representation logic. For all 128-bit ciphers, ciphertext bits are extracted as `c1 = X1[:, 128:]` and `c2 = X2[:, 128:]`, discarding the plaintext portion entirely. Feeding the plaintext difference $|P_1 - P_2|$ to any model would leak the fixed input difference ΔP, making every round trivially distinguishable regardless of the cipher's diffusion — an inflated accuracy that carries no cryptographic meaning. All five representations therefore operate exclusively on ciphertext bits.

Representation	Formula	Dimension	Notes
diff	|c1-c2|	n	XOR difference - direct differential signal
prod	c1*c2	n	Element-wise product
concat	[c1 | c2]	2n	Raw pair - model learns any joint function
stat	[| c1-c2| | HW/n]	n+1	**Recommended Default**
joint	[c1 | c2 | HW/n]	2n + 1	Concat augmented with HW scalar

NOTE: HW denotes the Hamming weight of the ciphertext difference, normalized by n:

HW(|c1 - c2|) / n.

The recommended default representation is `stat`. It is the most compact representation that carries both the differential signal and a global density measure. The XOR difference component directly encodes the differential distribution exploited by classical differential cryptanalysis. The normalised Hamming weight scalar provides a single statistic that summarises the weight of the output difference — particularly informative for SPN ciphers such as GIFT, PRESENT, and RECTANGLE, where S-box output distributions create a strong and consistent bias in the Hamming weight of the ciphertext

difference. `stat` is strictly more informative than `diff` alone at the cost of exactly one additional input dimension.

For the CNN, the primary tensor is additionally reshaped into a 2D spatial map of shape `(C, 8, 16)` before being passed to the convolutional layers, where `C = 1` for single-channel representations (`diff`, `prod`, `stat`) and `C = 2` for two-channel representations (`concat`, `joint`). The 8×16 layout aligns with nibble and byte boundaries of the target ciphers, allowing early convolutional filters to learn local bit-level differential patterns that correspond to S-box and word-level operations. The Hamming weight scalar is not reshaped — it is concatenated to the encoder output after the convolutional blocks, preserving its global nature. MLP and Siamese receive all representations as flat vectors.

The `prod` and `joint` representations are included for completeness. `prod` is consistently the weakest representation across all cipher families. `joint` offers marginal accuracy gains over `stat` at very low round counts but doubles the input dimensionality, making it less practical as a default.

5. Machine Learning Models

5.1 Multi-Layer Perceptron (MLP)

The MLP serves as a strong baseline. Our architecture processes flat bit-vector inputs and consists of four fully connected layers with batch normalization, ReLU activations, and dropout regularization.

Layer	Size	Activation	Regularization
Input	Dynamic (128/129/256/257 bits)	—	—
Hidden 1	512 units	ReLU	BatchNorm, Dropout 0.3
Hidden 2	256 units	ReLU	BatchNorm, Dropout 0.2
Hidden 3	128 units	ReLU	—
Output	1 unit	Sigmoid	—

Training uses AdamW optimizer (lr = 3e-4) with binary cross-entropy loss. Early stopping with patience 7 prevents overfitting. The default batch size is 1024.

5.2 Convolutional Neural Network (CNN)

The Convolutional Neural Networks architecture processes the pair (c1,c2) by first converting them into a structured spatial representation, which is then treated as an image-like tensor. Convolutional filters are applied to this representation to learn local patterns that reflect differential propagation and statistical dependencies present in the ciphertexts.

The model consists of multiple 2D convolutional blocks, each including a Conv2D layer, batch normalization, and GELU activation, followed by pooling layers that progressively reduce spatial dimensions while preserving salient features. The resulting feature maps are flattened and passed through fully connected layers to produce a compact feature vector.

An optional global statistical feature (Hamming weight) is concatenated with the learned representation before classification. The final classification head applies fully connected layers with nonlinear activation and a sigmoid output to perform binary decision making, consistent with a Distinguishing attack framework.

5.3 Siamese Network

The Siamese (twin) network architecture is naturally suited for pairwise comparison tasks. Both ciphertexts c1 and c2 are passed through a shared encoder network with tied weights, producing embedding vectors e1 and e2. Instead of combining the embeddings, a statistical feature vector is constructed from the ciphertext pair, including the Hamming weight of $\Delta C = C1 \text{ XOR } C2$, bit correlation frequencies, and empirical probability distributions. The feature vector is then fed into a classification head to predict whether the pair is a real differential pair or a random pair.

The weight-sharing enforces that the model learns a permutation-invariant embedding space for ciphertexts, where the classifier operates on relational features between the two ciphertexts. This is analogous to metric learning: real cipher pairs should be more distinguishable from random pairs in embedding space.

The shared encoder is a 4-layer MLP with residual connections. The combination function uses both absolute difference and product to capture both magnitude and directional similarity information.

5.4 Training Protocol

Hyperparameter	Value
Optimizer	AdamW
Learning Rate	3e-4
Batch Size	1024

Hyperparameter	Value
Max Epochs	40
Early Stopping Patience	8
Loss Function	BCE
Weight Decay	1e-4

6. Implementation

6.1 Code Architecture

The codebase is organized as follows:

- core/config.py — Global configuration: cipher list, round ranges, dataset parameters, device settings.
- core/utils.py — Shared cryptographic utilities: 64-bit rotation (rotr64), bit masking (MASK64), byte/integer conversion helpers.
- data/dataset.py — Dataset generation engine: plaintext sampling, ciphertext computation, labeling, and numpy serialization.
- ciphers/ — Cipher implementations: ascon.py, gift.py, and 10 additional cipher modules.
- tests/selftest.py — Known-answer tests for each cipher to validate implementations.
- generate_datasets.py — CLI driver for dataset generation with argparse interface.
- mlp_model.py — MLP model, training loop, and evaluation with three input modes.
- cnn_model.py — CNN architecture with multi-scale convolutional blocks.
- siamese_model.py — Siamese twin network with shared encoder.

6.2 Reproducing Results

To generate all datasets:

```
python generate_datasets.py
```

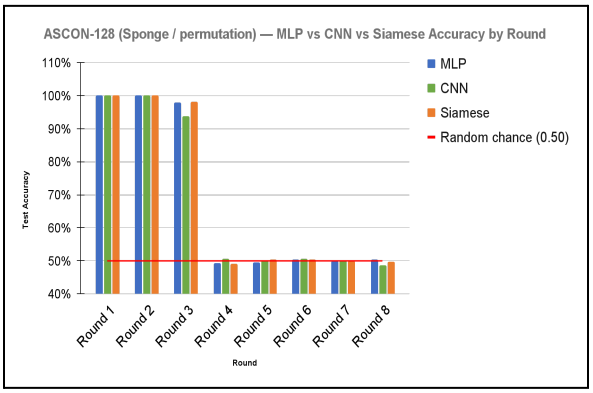
To train and evaluate the MLP distinguisher on GIFT with the diff representation:

```
python run.py -- model mlp --cipher gift --mode diff --rounds 3 4 5 6 7
```

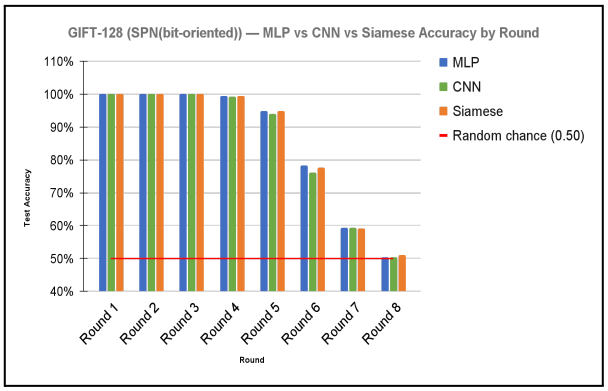
To train the CNN on all ciphers with background logging:

```
nohup python -u cnn_base.py --cipher all > cnn.log 2>&1
```

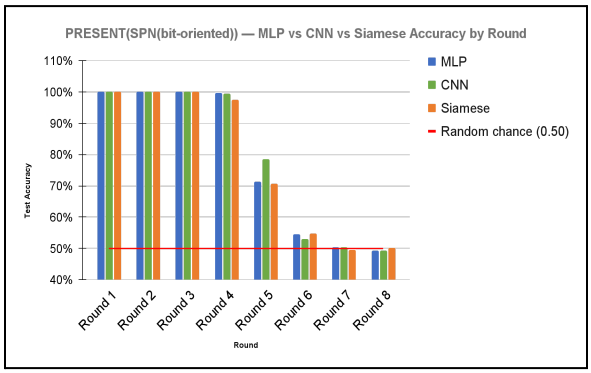
7. Results and Observations



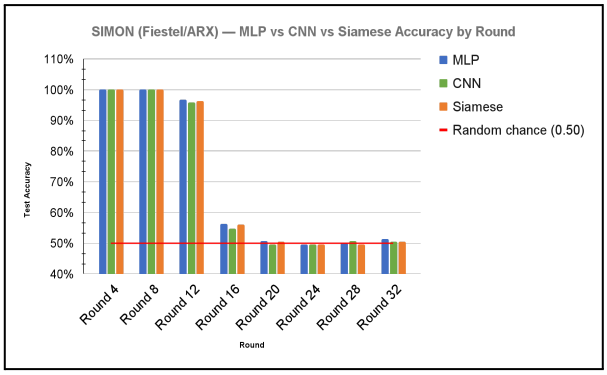
ASCON



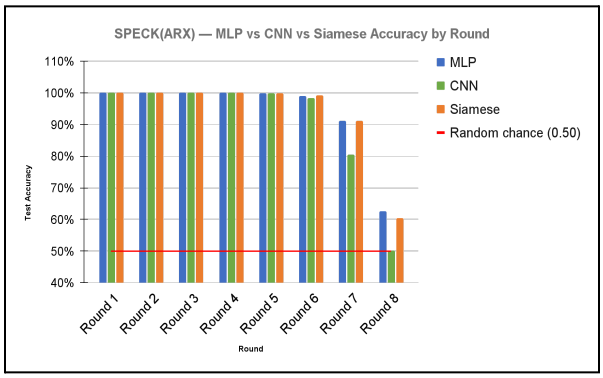
GIFT



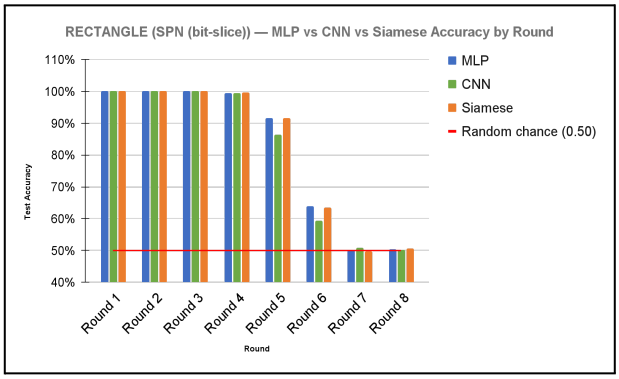
PRESENT



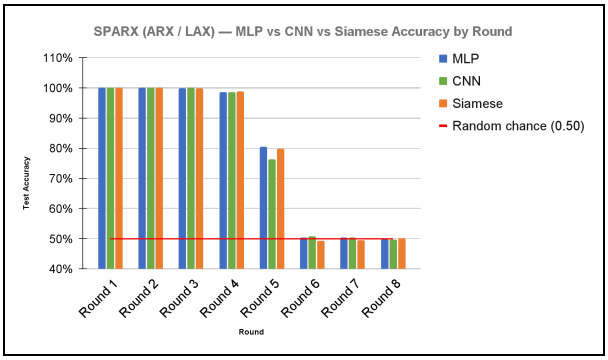
SIMON



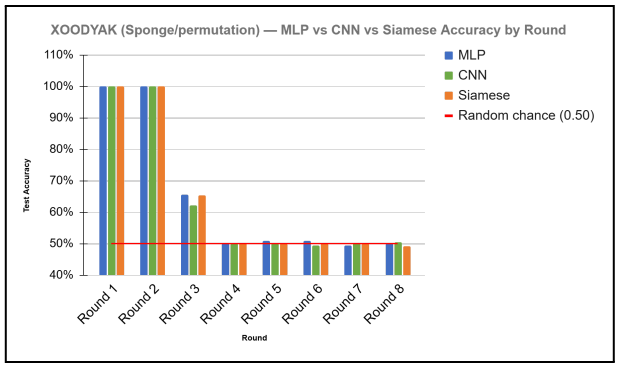
SPECK



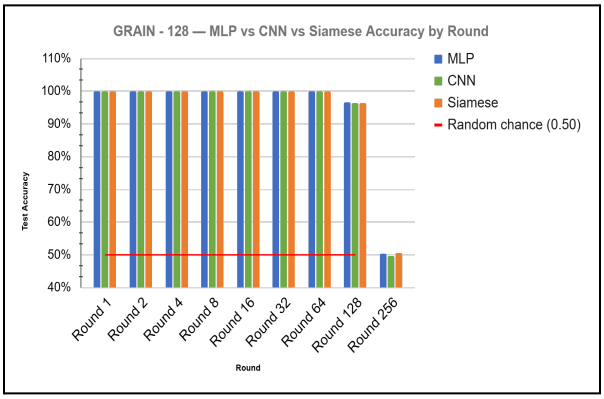
RECTANGLE



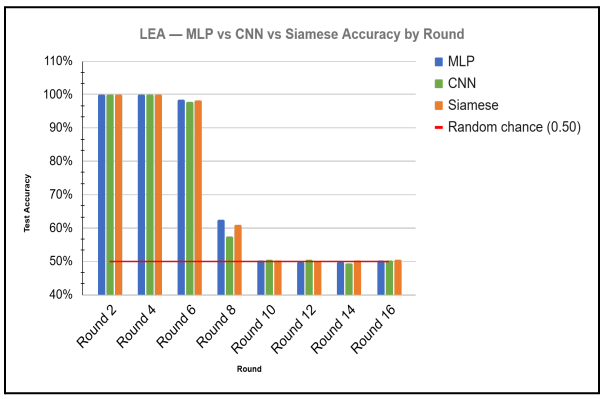
SPARX



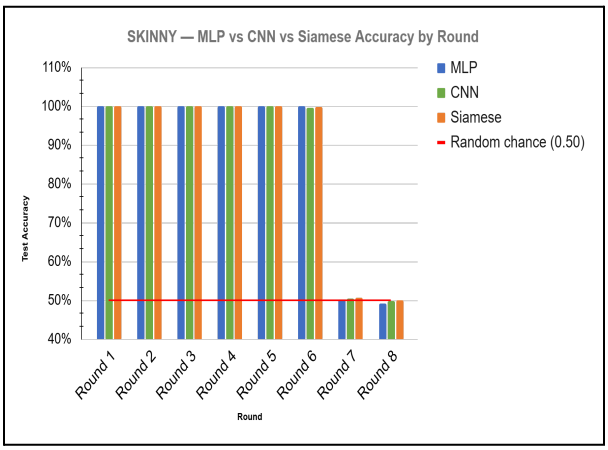
XOODYAK



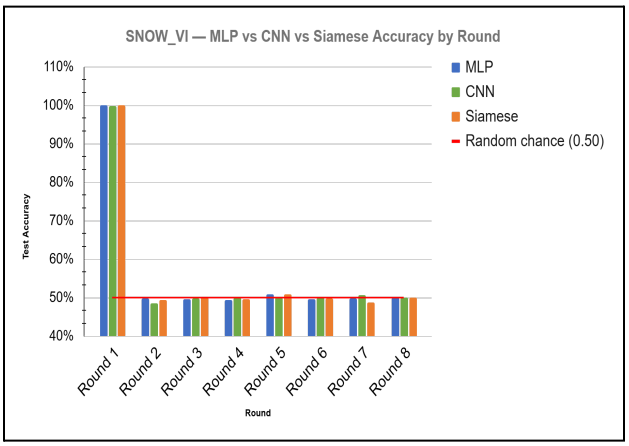
GRAIN - 128



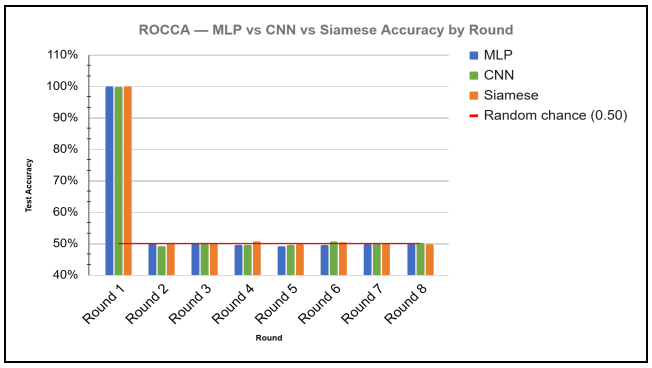
LEA



SKINNY



SNOW_VI



ROCCA

7.1 Key Observations

1. **Universal perfect accuracy at low rounds.** Every cipher across all design families achieves 1.000 accuracy at sufficiently low round counts, confirming that reduced-round variants are trivially distinguishable by neural networks regardless of architecture.
2. **Accuracy collapses sharply, not gradually.** The drop from high accuracy to random chance (~0.50) happens abruptly within one or two rounds for most ciphers, suggesting cipher diffusion is threshold-based rather than linear.
3. **All three models perform near-identically.** MLP, CNN, and Siamese networks differ by at most 1–2% across nearly all ciphers and rounds, meaning architecture choice is largely irrelevant — the distinguishing signal is either strongly present or completely absent.
4. **Sponge-based designs are the fastest to achieve randomness.** ROCCA and SNOW become indistinguishable from Round 2 onward; XOODYAK and ASCON by Round 4 — making sponge permutations the most effective at rapidly destroying detectable structure.
5. **GRAIN is the most resilient cipher overall.** It maintains perfect 1.000 accuracy through Round 64 and only collapses at Round 256, far outlasting all block ciphers, reflecting the unique nature of LFSR+NLFSR stream cipher structure.
6. **SKINNY is the most resilient block cipher.** It holds perfect accuracy through Round 6, breaking only at Round 7 — the latest breaking point among all block ciphers tested.
7. **CNN underperforms in the transition zone most frequently.** Cases include ASCON Round 3 (0.937 vs ~0.981), RECTANGLE Round 5 (0.863 vs ~0.916), and SPECK Round 8 (0.501 vs 0.626 for MLP) — suggesting CNN's spatial feature extraction loses relevance as outputs approach randomness.
8. **PRESENT Round 5 is the only case where CNN clearly leads**, achieving 0.786 versus MLP's 0.714 and Siamese's 0.706 — the sole instance across all 13 ciphers where CNN outperforms in the transition zone.
9. **ARX ciphers require more rounds in absolute terms before breaking**, with SIMON persisting to Round 16, LEA to Round 8, and SPECK to Round 8 —

consistent with ARX designs relying on rotation and XOR mixing rather than explicit S-boxes, which slows diffusion per round.

10. **Neural networks break all tested ciphers well before their full round specification.** No cipher is secure at its complete design round count once the models are given a partial-round variant — indicating these distinguishing attacks are practically relevant in reduced-round cryptanalysis contexts.

8. Conclusion

Observation	MLP	CNN	Siamese
Performance in transition zone	Strongest performer retains highest accuracy longest as rounds increase, e.g. 0.626 on SPECK Round 8	Weakest performer drops fastest in transition zone, e.g. only 0.501 on SPECK Round 8	Closely mirrors MLP, rarely deviating by more than 1–2% across ciphers
Sensitivity to diffusion onset	Degrades smoothly and predictably shows a consistent staircase-like decline across multi-round ciphers like GIFT-128 and SPECK	Most sensitive to diffusion onset exhibits the sharpest single-round drops, e.g. 1.000 to 0.937 on ASCON Round 3	Degrades steadily without erratic drops, closely tracking MLP's decline pattern across all cipher families
Cipher-specific strengths	No notable weak spots across any cipher family performs reliably on SPN, ARX, Sponge, and Stream designs	Uniquely leads on PRESENT Round 5 (0.786) the only cipher and round where CNN outperforms both other models	Marginally stronger on sponge-based ciphers scores 0.982 on ASCON Round 3 and 0.654 on XOODYAK Round 3, slightly above MLP and CNN

9.References

[1] Gohr, A. (2019). Improving Attacks on Round-Reduced Speck32/64 Using Deep Learning. In CRYPTO 2019.

[2] Biham, E., & Shamir, A. (1991). Differential Cryptanalysis of DES-like Cryptosystems. Journal of Cryptology, 4(1), 3–72.

[3] Beierle, C., et al. (2016). The SKINNY Family of Block Ciphers and Its Low-Latency Variant MANTIS. CRYPTO 2016.

[4] Banik, S., et al. (2017). GIFT: A Small Present. CHES 2017.

[5] Bogdanov, A., et al. (2007). PRESENT: An Ultra-Lightweight Block Cipher. CHES 2007.

[6] Bernstein, D. J., & Schwabe, P. (2012). NEON Crypto. CHES 2012.

[7] Baksi, A., et al. (2021). Machine Learning Assisted Differential Distinguishers for Lightweight Ciphers. DATE 2021.

[8] Hou, Z., et al. (2021). Cryptanalysis of Round-Reduced SIMON32 Based on Deep Learning. IEEE Access.

[9] Chen, Y., et al. (2022). Neural Aided Statistical Attack for Cryptanalysis. CCS 2022.

[10] ASCON Team. (2021). Ascon v1.2. NIST Lightweight Cryptography Finalist.

Github link : https://github.com/Vyash2002/AC_Project